

Smart Contract Security Audit Report

Table Of Contents

1 Executive Summary

2 Audit Methodology

3 Project Overview

3.1 Project Introduction

3.2 Vulnerability Information

4 Code Overview

4.1 Contracts Description

4.2 Visibility Description

4.3 Vulnerability Summary

5 Audit Result

6 Statement

1 Executive Summary

On 2026.10.27, the SlowMist security team received the Changer+ team's security audit application for Changer+, developed the audit plan according to the agreement of both parties and the characteristics of the project, and finally issued the security audit report.

The SlowMist security team adopts the strategy of "white box lead, black, grey box assists" to conduct a complete security test on the project in the way closest to the real attack.

The test method information:

Test method	Description
Black box testing	Conduct security tests from an attacker's perspective externally.
Grey box testing	Conduct security testing on code modules through the scripting tool, observing the internal running status, mining weaknesses.
White box testing	Based on the open source code, non-open source code, to detect whether there are vulnerabilities in programs such as nodes, SDK, etc.

The vulnerability severity level information:

Level	Description
Critical	Critical severity vulnerabilities will have a significant impact on the security of the DeFi project, and it is strongly recommended to fix the critical vulnerabilities.
High	High severity vulnerabilities will affect the normal operation of the DeFi project. It is strongly recommended to fix high-risk vulnerabilities.
Medium	Medium severity vulnerability will affect the operation of the DeFi project. It is recommended to fix medium-risk vulnerabilities.
Low	Low severity vulnerabilities may affect the operation of the DeFi project in certain scenarios. It is suggested that the project team should evaluate and consider whether these vulnerabilities need to be fixed.
Weakness	There are safety risks theoretically, but it is extremely difficult to reproduce in engineering.

Level	Description
Suggestion	There are better practices for coding or architecture.

2 Audit Methodology

The security audit process of SlowMist security team for smart contract includes two steps:

- Smart contract codes are scanned/tested for commonly known and more specific vulnerabilities using automated analysis tools.
- Manual audit of the codes for security issues. The contracts are manually analyzed to look for any potential problems.

Following is the list of commonly known vulnerabilities that was considered during the audit of the smart contract:

Serial Number	Audit Class	Audit Subclass
1	Overflow Audit	-
2	Reentrancy Attack Audit	-
3	Replay Attack Audit	-
4	Flashloan Attack Audit	-
5	Race Conditions Audit	Reordering Attack Audit
6	Permission Vulnerability Audit	Access Control Audit
		Excessive Authority Audit
7	Security Design Audit	External Module Safe Use Audit
		Compiler Version Security Audit

Serial Number	Audit Class	Audit Subclass
7	Security Design Audit	Hard-coded Address Security Audit
		Fallback Function Safe Use Audit
		Show Coding Security Audit
		Function Return Value Security Audit
		External Call Function Security Audit
		Block data Dependence Security Audit
		tx.origin Authentication Security Audit
8	Denial of Service Audit	-
9	Gas Optimization Audit	-
10	Design Logic Audit	-
11	Variable Coverage Vulnerability Audit	-
12	"False Top-up" Vulnerability Audit	-
13	Scoping and Declarations Audit	-
14	Malicious Event Log Audit	-
15	Arithmetic Accuracy Deviation Audit	-
16	Uninitialized Storage Pointer Audit	-

3 Project Overview

3.1 Project Introduction

The project under audit is a suite of multi-chain token transfer relayer contracts supporting EVM, TRON, and Solana. After users grant the necessary approvals and sign transfer requests, designated relayer operators submit transactions on their behalf. The contracts support transfers to multiple recipients and batch processing.

3.2 Vulnerability Information

The following is the status of the vulnerabilities found in this audit:

NO	Title	Category	Level	Status
N1	Potential Risks When Relaying Malicious Tokens	Others	Suggestion	Acknowledged
N2	Request Expiration Validation Cannot Restrict the Actual Signature Creation Time	Others	Information	Acknowledged
N3	Potential Risks of Incompatible Token Return Values	Design Logic Audit	Suggestion	Acknowledged
N4	Unused Error Definitions in RelayerError	Others	Suggestion	Acknowledged

4 Code Overview

4.1 Contracts Description

Audit Version:

smart_contract.zip: c465279ec91cc9fb25df70f4c16c8fc614677195be22eb46b19aea0bb2ab8b58

Audit Scope:

```
.
├── evm
│   ├── WorkerManager.sol
│   └── tokenTransfer
│       ├── ApproveTransferRelayer.sol
│       ├── IApproveTransferRelayer.sol
│       ├── IPermitTransferRelayer.sol
│       └── PermitTransferRelayer.sol
├── solana
│   ├── constants.rs
│   ├── contexts.rs
│   ├── ed25519.rs
│   ├── errors.rs
│   ├── hash.rs
│   ├── lib.rs
│   ├── nonce.rs
│   ├── state.rs
│   ├── token_policy.rs
│   └── validation.rs
└── tron
    ├── Migrations.sol
    ├── token
    │   ├── ITRC20.sol
    │   └── SafeTRC20.sol
    ├── transfer
    │   ├── ITokenTransferRelayer.sol
    │   └── TokenTransferRelayer.sol
    └── utils
        ├── Context.sol
        ├── EIP712.sol
        ├── Ownable.sol
        └── WorkerManager.sol
```

The main network address of the contract is as follows:

Address	Chain
5eRYiqJMS6SM5HB7CKbRYnXuDg6Whks5hXZNG3o2r792	Solana
0x5498229Ad9e5C79094ec1ec08c800D14929D08a8	BNB Smart Chain

Address	Chain
0x3BD01fcbE46aB949e1f70A129831BC1C2937128a	Ethereum
0x0a90e94250f15B256CeBa7A6633068f0BAD6F193	Ethereum
TBE9E9TreLZiyfrGXULzLEkVqKaJT7ZgMt	TRON

The source code is not public. The deployed bytecode and Solana program binary have been verified to match the audited version.

4.2 Visibility Description

The SlowMist Security team analyzed the visibility of major contracts during the audit, the result as follows:

WorkerManager			
Function Name	Visibility	Mutability	Modifiers
addWorkers	Public	Can Modify State	onlyOwner
addWorker	Public	Can Modify State	onlyOwner
_addWorker	Internal	Can Modify State	-
removeWorker	Public	Can Modify State	onlyOwner
_removeWorker	Internal	Can Modify State	-
renounceOwnership	Public	-	-
isWorker	Public	-	-

ApproveTransferRelayer			
Function Name	Visibility	Mutability	Modifiers

ApproveTransferRelayer			
<Constructor>	Public	Can Modify State	Ownable EIP712
verify	External	-	-
transferMultiple	External	Can Modify State	onlyWorker
_callTransfer	Internal	Can Modify State	-
transfer	External	Can Modify State	onlyWorker nonReentrant
getEncoded	Public	-	-
_verifyNonce	Internal	-	-
_validateRequest	Internal	-	-
_requestHash	Internal	-	-
_errorDetails	Internal	-	-
_verifySig	Internal	-	-
getNonce	External	-	-
cancelCurrentNonce	External	Can Modify State	-
getBalances	External	-	-

PermitTransferRelayer			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	Ownable EIP712
verify	External	-	-
transferMultiple	External	Can Modify State	onlyWorker
_callTransfer	Internal	Can Modify State	-

PermitTransferRelayer			
transfer	External	Can Modify State	onlyWorker nonReentrant
encode	Public	-	-
_verifyNonce	Internal	-	-
_validateRequest	Internal	-	-
_requestHash	Internal	-	-
_errorDetails	Internal	-	-
_verify	Internal	-	-
getNonce	External	-	-
cancelCurrentNonce	External	Can Modify State	-

SafeTRC20			
Function Name	Visibility	Mutability	Modifiers
safeTransfer	Internal	Can Modify State	-
safeTransferFrom	Internal	Can Modify State	-
safeIncreaseAllowance	Internal	Can Modify State	-
safeDecreaseAllowance	Internal	Can Modify State	-
forceApprove	Internal	Can Modify State	-
_callOptionalReturn	Private	Can Modify State	-
_callOptionalReturnBool	Private	Can Modify State	-

TokenTransferRelayer			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	Ownable EIP712
verify	External	-	-
transferMultiple	External	Can Modify State	onlyWorker
_callTransfer	Internal	Can Modify State	-
transfer	External	Can Modify State	onlyWorker nonReentrant
encode	Public	-	-
_verify	Internal	-	-
_recover	Private	-	-
_verifyNonce	Internal	-	-
_validateRequest	Internal	-	-
_requestHash	Internal	-	-
_errorDetails	Internal	-	-
getNonce	External	-	-
cancelCurrentNonce	External	Can Modify State	-
rescueTRX	External	Can Modify State	onlyOwner nonReentrant
rescueTRC20	External	Can Modify State	onlyOwner nonReentrant
getBalances	External	-	-

WorkerManager			
Function Name	Visibility	Mutability	Modifiers

WorkerManager			
addWorkers	Public	Can Modify State	onlyOwner
addWorker	Public	Can Modify State	onlyOwner
_addWorker	Internal	Can Modify State	-
removeWorker	Public	Can Modify State	onlyOwner
_removeWorker	Internal	Can Modify State	-
isWorker	Public	-	-

token_transfer_relayer			
Function	Visibility	Module	Caller
initialize	pub	token_transfer_relayer	Upgrade authority + initial owner + payer
set_worker	pub	token_transfer_relayer	owner
propose_owner	pub	token_transfer_relayer	owner
accept_owner	pub	token_transfer_relayer	pending new owner
transfer	pub	token_transfer_relayer	worker + from (both signers; requires a signed request)
close_expired_nonce_v3	pub	token_transfer_relayer	Anyone
close_used_nonce	pub	token_transfer_relayer	Anyone

4.3 Vulnerability Summary

[N1] [Suggestion] Potential Risks When Relaying Malicious Tokens

Category: Others

Content

In ApproveTransferRelayer.sol, PermitTransferRelayer.sol, and TokenTransferRelayer.sol, the transferMultiple function uses delegatecall to itself to execute user-signed requests individually. While the outer call can record failures and continue processing when individual subcalls fail, these calls do not enforce gas limits for each request or explicitly reserve sufficient gas for subsequent processing by the outer call.

After signature and nonce verification, the contract interacts with the token specified in the user's request. If an attacker controls the token contract and signs a valid request, and the worker includes this request in a batch alongside legitimate requests from other users, the malicious token can deliberately consume excessive gas during execution. If the remaining gas is insufficient for the outer call to complete error handling, event emission, or processing of subsequent requests, the entire batch may revert.

Code location:

evm/tokenTransfer/ApproveTransferRelayer.sol#L47

```
function transferMultiple(
    ApproveTransferRequest[] calldata requests,
    bytes[] calldata signatures
) external onlyWorker
returns (bool[] memory)
{
    ...
        (bool success, bytes memory returnData) = _callTransfer(data);
        ...
}

function _callTransfer(bytes memory callData) internal returns (bool, bytes memory) {
    return address(this).delegatecall(callData);
}
```

evm/tokenTransfer/PermitTransferRelayer.sol#L48

```
function transferMultiple(
    PermitTransferRequest[] calldata requests,
    bytes[] calldata signatures
) external onlyWorker
returns (bool[] memory)
{
  ...
        (bool success, bytes memory returnData) = _callTransfer(data);
       ...
        }
    }
    return results;
}

function _callTransfer(bytes memory callData) internal returns (bool, bytes memory) {
    return address(this).delegatecall(callData);
}
```

tron/transfer/TokenTransferRelayer.sol#L48

```
function transferMultiple(
    TokenTransferRequest[] calldata requests,
    bytes[] calldata signatures
) external onlyWorker
returns (bool[] memory)
{
    ...
        (bool success, bytes memory returnData) = _callTransfer(data);
        ...
    }
    return results;
}

function _callTransfer(bytes memory callData) internal returns (bool, bytes memory) {
    // delegatecall preserves the original worker as msg.sender while each
    // transfer keeps its own reentrancy guard and rollback boundary.
    return address(this).delegatecall(callData);
}
```

Solution

Apply appropriate gas limits to each subcall and reserve sufficient gas for the outer call to handle return values, emit events, and process remaining requests. Additionally, enforce limits on batch sizes and the amount of return data copied.

Status

Acknowledged; Following discussions with the project team, the team confirmed that a token whitelist will be enforced off-chain, and only supported tokens will be processed.

[N2] [Information] Request Expiration Validation Cannot Restrict the Actual Signature Creation Time

Category: Others

Content

In ApproveTransferRelayer.sol, PermitTransferRelayer.sol, and TokenTransferRelayer.sol, the validUntilTime field is included in the signed request. During execution, the contracts verify that the current timestamp has not exceeded the expiration time and that the expiration time is no more than 90 minutes in the future.

However, the contracts cannot determine when the signature was actually created. A user may sign a request several days in advance with a future expiration time. Although the request initially cannot be executed because its expiration time is too far in the future, it becomes executable within the 90-minute window before expiration, provided that the signature, nonce, token approvals, and other execution conditions remain valid. It can then be executed by the worker.

Code location:

evm/tokenTransfer/ApproveTransferRelayer.sol#L139

evm/tokenTransfer/PermitTransferRelayer.sol#L154

tron/transfer/TokenTransferRelayer.sol#L180

```
function _validateRequest(ApproveTransferRequest calldata request) internal view {
    require(request.validUntilTime != 0 && block.timestamp <= request.validUntilTime,
"TFR: request expired");
```

```
    require(request.validUntilTime <= block.timestamp + MAX_VALIDITY_WINDOW, "TFR:
validity too long");
    require(request.value > 0, "TFR: value invalid");
    require(request.data.length > 0, "TFR: empty data");
    ...
}
```

Solution

N/A

Status

Acknowledged

[N3] [Suggestion] Potential Risks of Incompatible Token Return Values

Category: Design Logic Audit

Content

In TokenTransferRelayer.sol, the rescueTRC20 function allows the owner to withdraw TRC-20 tokens held by the relayer contract. This function uses safeTransfer from SafeTRC20.sol to invoke the token's transfer function, which treats any non-empty return value other than true as a failure and reverts the transaction.

However, TRON USDT's transfer function returns false even when the transfer succeeds. Therefore, if USDT is transferred to the relayer contract, an attempt by the owner to withdraw it through rescueTRC20 will revert due to the false return value, even if the underlying token transfer has been executed. As a result, such tokens cannot be recovered through the existing interface.

Code location: tron/token/SafeTRC20.sol#L88

```
function _callOptionalReturn(ITRC20 token, bytes memory data) private {
    ...

    if (returnSize == 0 ? address(token).code.length == 0 : returnValue != 1) {
        revert SafeTRC20FailedOperation(address(token));
    }
}
```

Solution

Implement a restricted token withdrawal adapter specifically for the verified TRON USDT contract address exhibiting this return-value behavior.

Status

Acknowledged; Following discussions with the project team, the team stated that these functions are not required and that they have no plans to handle the recovery of tokens mistakenly transferred by users in the future.

[N4] [Suggestion] Unused Error Definitions in RelayerError

Category: Others

Content

In errors.rs, the RelayerError enum is used to define the program's custom errors. The current enum includes FromAccountMismatch, RecipientOwnerMismatch, DataInvalid, InsufficientAllowance, ValidUntilTimeInvalid, and UnexpectedTokenBalance. However, no execution paths in the provided source code return these errors, and their deprecated status or intended purpose is not individually documented. These unused definitions increase the maintenance burden of the error interface.

Code location: solana/errors.rs

```
#[error_code]
pub enum RelayerError {
    // ...
    #[msg("From account mismatch")]
    FromAccountMismatch,
    #[msg("Recipient owner account mismatch")]
    RecipientOwnerMismatch,
    // ...
    #[msg("Invalid transfer data")]
    DataInvalid,
    // ...
    #[msg("Insufficient delegated allowance")]
    InsufficientAllowance,
    // ...
    #[msg("Invalid valid_until_time")]
```

```
    ValidUntilTimeInvalid,
    // ...
    #[msg("Transfer did not credit the exact signed amount")]
    UnexpectedTokenBalance,
    // ...
}
```

Solution

If the program has already been deployed or existing clients depend on the current error codes, it is recommended to preserve the names and order of these variants and add a comment to each indicating that it is "currently unused and retained for error code compatibility." If the program has not yet been deployed and no compatibility dependencies exist, the unused variants may be removed.

Status

Acknowledged; Following discussions with the project team, the team stated that this issue does not pose a risk to the protocol and decided not to take any further action.

5 Audit Result

Audit Number	Audit Team	Audit Date	Audit Result
0X002610290001	SlowMist Security Team	2026.10.27 - 2026.10.29	Passed

Summary conclusion: The SlowMist security team audited the project using a combination of manual review and SlowMist's analysis tool. During the audit, we identified three suggestions, and one informational finding. All findings were acknowledged.

6 Statement

SlowMist issues this report with reference to the facts that have occurred or existed before the issuance of this report, and only assumes corresponding responsibility based on these.

For the facts that occurred or existed after the issuance, SlowMist is not able to judge the security status of this project, and is not responsible for them. The security audit analysis and other contents of this report are based on the documents and materials provided to SlowMist by the information provider till the date of the insurance report (referred to as "provided information"). SlowMist assumes: The information provided is not missing, tampered with, deleted or concealed. If the information provided is missing, tampered with, deleted, concealed, or inconsistent with the actual situation, the SlowMist shall not be liable for any loss or adverse effect resulting therefrom. SlowMist only conducts the agreed security audit on the security situation of the project and issues this report. SlowMist is not responsible for the background and other conditions of the project.



Official Website

www.slowmist.com



E-mail

team@slowmist.com



Twitter

@SlowMist_Team



Github

https://github.com/slowmist